

CAMPUSFEST — ARQUITECTURA EN CAPAS

VERSIÓN 1.0

Diagrama de Arquitectura de Software & Especificación de Componentes

Fecha: 8 de Agosto, 2026

- 1. Presentation ● 2. API / Application ● 3. Services / Business ● 4. Domain / Models ● 5. Persistence
● 6. Infrastructure

CAPA 1

Presentation Layer (Frontend)

HTML5 | CSS3 (Bootstrap 5) | JS (ES6+)

Public Site Pages

- Home & Landing (/client/index.html)
- Activities Listing & Filter
- Activity Detail View
- Registration Form

Admin UI Panel

- Dashboard & Stand Management
- Participant List & Waitlist View
- Admin Auth Form (x-admin-key)
- Capacity Overrides

↓ Fetch API / Protocol: HTTPS (Port 443 / Dev: 5000) — JSON Payload ↓

CAPA 2

API / Application Layer

Node.js | Express Framework

Router / Gateway

/api/actividades
/api/inscripciones
/api/auth

Controllers

activityController
participantController
registrationController
standController, authController

Middlewares

- Sanitization & Validation
- Auth (x-admin-key)
- Centralized Error Handler

↓ Internal Method Calls (Controller → Service) ↓

CAPA 3

Services / Business Logic Layer

Pure JS / Domain Services

registrationService

- Capacity Check & Spot Allocation
- Duplicate Registration Prevention
- Waitlist FIFO Enqueue & Promotion

activityService & StandService

- Activity Scheduling & Status
- Stand Reservation Logic
- Minimal Edits & Idempotency Rules

↓ Mongoose ODM Query Execution ↓

CAPA 4

Domain / Models Layer

Mongoose ODM

Activity

Fields: title, capacity, spots.
Index: date: 1

Participant

Fields: email, name, phone.
Index: email: 1 (unique)

Registration

Fields: status, date.
Unique: (partId + actId)

Stand

Fields: number, location.
Index: number: 1

↓ Wire Protocol (MongoDB Driver / TLS) ↓

CAPA 5

Data Access / Persistence Layer

MongoDB Atlas Cluster

Database: CampusFestDB

- Collections: activities, participants, registrations, stands
- Connection: Secure string from process.env via MONGODB_URI placeholder
- Primary Indexes: Compound unique index on registrations(participantId, activityId)

↓ System Integration & Support Services ↓

CAPA 6

Infrastructure / Integrations

Node Runtime & Cross-Services

CORS & Static

Express static serving from / client

dotenv

Environment vars loader (.env)

Email Provider

Nodemailer / SMTP (Placeholder)

Logging

Console logger + expandable metrics

ASPECTOS TRANSVERSALES

- **Autenticación Admin:** Header x-admin-key validado contra ADMIN_PASSWORD.
- **Sanitización de Entradas:** Limpieza de HTML/Scripts previniendo XSS.
- **Validación:** Schema-level checks en Middleware y Mongoose.
- **Seguridad de Secretos:** Cero credenciales en código source; uso de .env / .env.example.
- **Manejo Estándar de Errores:** Formato JSON unificado:

```
{ success: false, message: "..."} 
```

ANOTACIONES DE NEGOCIO

⚠ **Regla: Límite de Capacidad**
Si availableSpots == 0, la inscripción asigna automáticamente el estado WAITLIST.

🚫 **Regla: No Duplicados**
Un índice único en MongoDB impide múltiples registros activos del mismo participante en la misma actividad.

🔄 **Política de Idempotencia**
Operaciones de consulta y actualización parcial evitan efectos secundarios ante reintentos de peticiones HTTP.

🔔 **Extensión Futura**
Hook en registrationService para invocación asíncrona del proveedor de correo (emails de confirmación).

NOTAS NO FUNCIONALES

- **Escalabilidad:** API sin estado (stateless) para fácil balanceo; DB como única fuente de verdad.
- **Observabilidad:** Logs en consola con estructura extendible para Prometheus/ Winston.
- **Rendimiento:** Índices B-Tree en campos de alta consulta (date, email).

- **Respaldos:** Backups automáticos gestionados por MongoDB Atlas.

CAMPUSFEST — ESPECIFICACIÓN DE CAPAS Y FLUJOS

Documentación Detallada de Endpoints, Lógica de Negocio y Flujos de Datos

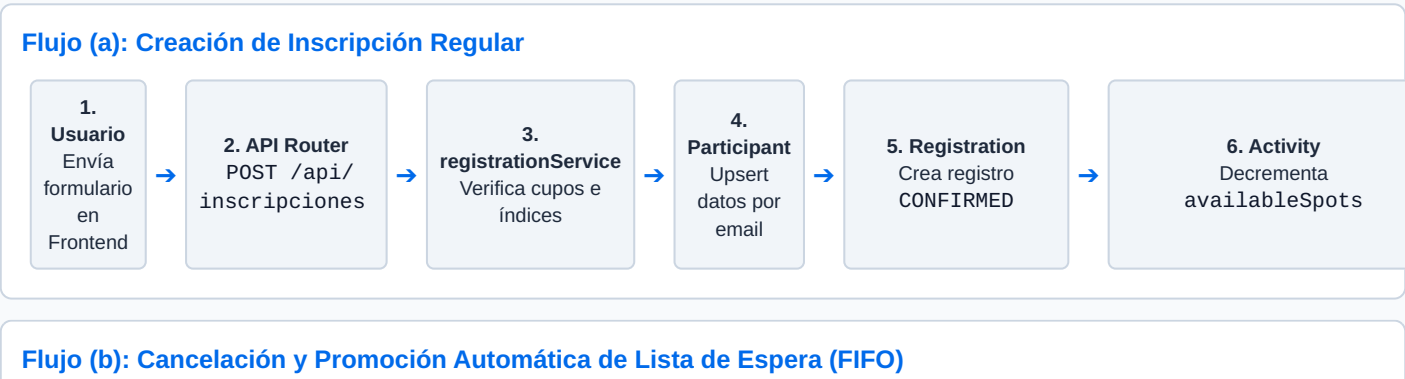
1. LEYENDA TEXTUAL EXPLICATIVA DE LAS CAPAS

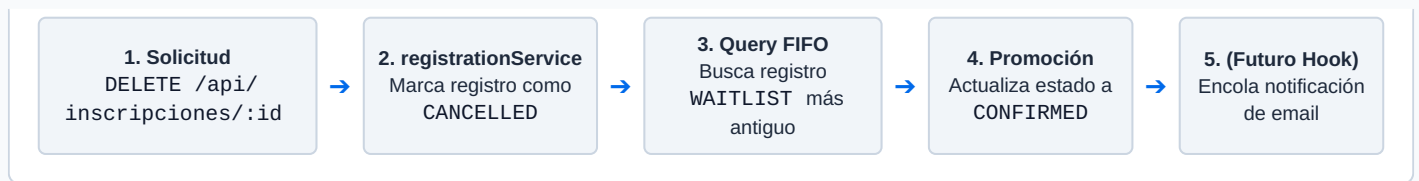
Capa	Descripción y Responsabilidades
1. Presentation Layer	Interfaz de usuario construida con HTML5, Bootstrap 5 y JavaScript vanilla ES6+. Responsable de la renderización del catálogo de actividades, formularios interactivos y la interfaz del administrador. Se comunica de forma asíncrona mediante <code>Fetch API</code> consumiendo endpoints REST over HTTPS.
2. API / Application Layer	Servidor Web Node.js + Express. Maneja el enrutamiento de peticiones HTTP, middleware de sanitización de inputs, validación de esquemas y autenticación administrativa mediante el header <code>x-admin-key</code> . Retorna respuestas normalizadas en formato JSON.
3. Services / Business Logic	Capa donde residen las reglas del dominio de CampusFest. Evalúa disponibilidad de cupos, asignación de cola de espera en esquema FIFO (First In, First Out), promoción automática de participantes al cancelar registros e idempotencia en operaciones.
4. Domain / Models Layer	Definición de esquemas de datos utilizando Mongoose ODM. Incluye las entidades principales (<code>Activity</code> , <code>Participant</code> , <code>Registration</code> , <code>Stand</code>) y la configuración de índices de rendimiento y restricciones de unicidad.
5. Data Access / Persistence	Cluster MongoDB Atlas. Garantiza la persistencia duradera, consistencia de datos y ejecución optimizada de consultas mediante índices compuestos y primarios. Carga de credenciales mediante variables de entorno local/cloud.
6. Infrastructure / Integrations	Componentes transversales del sistema: configuración CORS, servicio de archivos estáticos desde la carpeta <code>/client</code> , cargador <code>dotenv</code> , logs de sistema y conectores para futuras integraciones (p. ej. Nodemailer para envío de emails).

2. CATÁLOGO DE ENDPOINTS PRINCIPALES

Endpoint	Método	Acceso	Descripción y Comportamiento
<code>/api/actividades</code>	GET	Público	Retorna el catálogo completo de actividades activas, incluyendo fechas, ubicación y cupos disponibles (<code>availableSpots</code>).
<code>/api/inscripciones</code>	POST	Público	Procesa la solicitud de inscripción. Aplica upsert al participante por email, verifica disponibilidad y crea la reserva en estado <code>CONFIRMED</code> o <code>WAITLIST</code> .
<code>/api/inscripciones/lista-espera</code>	GET	Admin	Requiere header <code>x-admin-key</code> . Retorna el listado de participantes en cola FIFO ordenados por fecha de creación para actividades con cupo lleno.
<code>/api/auth/login</code>	POST	Público	Valida la clave administrativa provista contra la variable <code>ADMIN_PASSWORD</code> configurada en el servidor.

3. DIAGRAMAS DE FLUJO DE DATOS PRINCIPALES





Flujo (c): Flujos de Administración Protegidos

